

Operating Systems: Three Easy Pieces

Andrea and Remzi Arpaci-Dusseau

September 27, 2026

Contents

0.1	Introduction to Operating Systems	3
4	The Abstraction: The Process	5
4.1	Process Creation	5
4.2	Process States	5
4.3	Data Structures	6
5	Interlude: Process API	8
5.1	The <code>fork()</code> System Call	8
5.2	The <code>wait()</code> System Call	8
5.3	The <code>exec()</code> System Call	8
5.4	Motivating the API	8
5.5	Process Control and Users	9
6	Mechanism: Limited Direct Execution	9
6.1	Basic Technique: Limited Direct Execution	9
6.2	Problem #1: Restricted Operations	10
6.3	Problem #2: Switching Between Processes	11
6.3.1	Cooperative approach: Wait for syscalls	11
6.4	Non-cooperative approach	12
6.4.1	Saving and Restoring Context	12
13	The Abstraction: Address Spaces	13
13.1	Early Systems	13
13.2	Multiprogramming and Time Sharing	13
13.3	The Address Space	13
13.4	Goals	14
14	Interlude: Memory API	14
14.1	Types of Memory	14
14.2	The <code>malloc()</code> Call	14
14.3	The <code>free()</code> Call	14
14.4	Underlying OS Support	15
15	Mechanism: Address Translation	15
15.1	Assumptions	15
15.2	Example	15
15.3	Dynamic Relocation	16
15.4	Hardware support: A summary	17

15.5 Operating System Issues	17
16 Segmentation	17
16.1 Segmentation: Generalized Base/Bounds	17
16.2 Which segment are we referring to?	18
16.3 What about the stack?	18
16.4 Support for sharing	18
16.5 Fine-grained vs Coarse-grained segmentation	18
16.6 OS Support	19
17 Free-Space Management	19
17.1 Assumptions	19
17.2 Low-level mechanisms	19
17.2.1 Splitting and Coalescing	19
17.2.2 Tracking the size of allocated regions	20
17.2.3 Embedding a free list	21
Order: 16, 17, 18, 19, 20, 21, 22, 23, 26, 27, 28, 29, 30, 31, 32, 33, 7, 8, 10, 9, 14, 36, 39, 40, 42,	
37, 44, 45, 38, 41, 43, 47, 48, 49, 50, 51	

0.1 Introduction to Operating Systems

- Crux of the problem: How does the operating system virtualize resources?
 - What mechanisms and policies are implemented by the OS to attain virtualization?
 - How does the OS do this efficiently?
 - What hardware support is needed?

Definition 0.1 (Virtualization). The process in which the operating system takes a physical resource and abstracts it into an easier-to-use virtual form.

Definition 0.2 (System call). An interface or API exposed to the user to tell the OS what to do. A typical OS will export on the order of hundreds of system calls (syscall)

- The OS is also responsible for managing resources, each of the CPU, memory, and disk.

```
1 int main(int argc, char *argv[]) {
2     if (argc != 2) {
3         fprintf(stderr, "usage: %cpu<string>\n");
4         exit(1);
5     }
6     char *str = argv[1];
7     for (;;) {
8         Spin(1);
9         printf("%s\n", str);
10    }
11    return 0;
12 }
```

- The above code will simply just print the string that the user passed, forever.
- Interestingly though, even on a single core system we can run this program in parallel and all programs will run at the same time.
- The OS will virtualize each physical CPU into a seemingly infinite number of virtual CPUs, giving the illusion as if there are more than one.
- The OS policies defines the behaviour of the OS, including things like what programs get priority and what actions are permitted.
- A similar idea is applied with memory: memory is just an array of bytes.
- To read/write to memory, one needs to supply the address of whatever you want to access.
- Consider a program that allocates some memory, prints the address of the memory `p` and writes the number 0 into the first slot of the allocated memory. Finally it loops and increments the value stored at the address held in `p`.
- Similar to the example from before, if we run multiple instances of this program, then each instance will have allocated memory at the exact same address and each are updating the value at that address independently.
- This is an example of the OS virtualizing memory; each process has it's own private virtual address space.
- It is the OS's responsibility to map the virtual memory of each process to some portion of physical memory.
- Persistence: The OS is also responsible for ensuring that data that is stored in a volatile manner (such as DRAM) is not lost when power is lost or the system crashes. This is done through the usage of input/output, or I/O devices.

- Common IO devices in modern computers are hard drives or SSDs.
- Unlike the abstractions the OS performs on the physical CPU(s) and memory, there is no virtualization performed for each application.
- It is assumed that the user wants to share the contents of a file; one application may be used to create and edit a file and another to compile.
- The software that is responsible for managing and storing files on the disk is called the file system.

4 The Abstraction: The Process

- The crux of the problem: How to provide the illusion of many CPUs? In a system where there are only a few physical CPUs available, how can the OS provide the illusion of a nearly-endless supply of CPUs?
- The basic idea is simple: On a given CPU, the OS will run one process, then stop it to run another process, and so on. This is known as **Time sharing**.
- Using this the OS can provide the illusion that one CPU is running multiple processes, at the cost of slower performance the more processes need to be shared on one CPU.
- Mechanisms: low-level methods or protocols that implement functionality.
- Policies: higher-level algorithms that are responsible for making some kind of decision within an OS.
- A process at a given time can affect the memory that is reserved for it (its address space), as well as registers.
- Special registers include the Program Counter/Instruction pointer (PC/IP), which tells what the next instruction to be executed is and a stack pointer is used to manage the stack for function parameters, local variables and return values.
- Programs may also affect persistent storage devices as well.
- The OS must expose the following APIs for processes:
 1. Create: An OS must be able to create a new process.
 2. Destroy: An OS must be able to destroy an existing process.
 3. Wait: It is sometimes useful to wait for a specific process to stop
 4. Misc. control: Some way to temporarily halt and resume a process
 5. Status: Read-only APIs that give status info or some other pieces of metadata on a process.

4.1 Process Creation

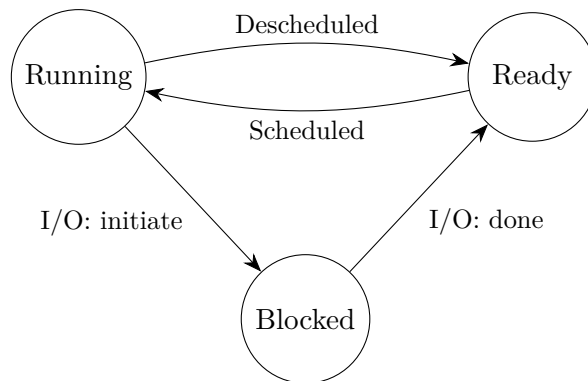
- The first thing the OS does is load the program code and any static data into memory (the address space of the process)
- This is done lazily; the OS will only fetch pieces of code or data that is necessary (opposed to eagerly). This is done to further support paging and swapping.
- The OS then allocates memory on the program's stack as well as the heap.
- It will then perform any remaining initialization tasks, such as those pertaining to IO. The OS then transfers control of the CPU to the newly-created process.

4.2 Process States

In a simplified view, a process can be in one of three states:

1. Running: The process is currently being executed.
2. Ready: A process is ready to run but is not currently being executed for some reason from the OS.

3. Blocked: A process has performed some operation in which it needs to wait for in order to proceed. An example is an IO request, in which the process needs to wait for the data to be fetched and is thus blocked; in the meantime some other process is free to use the processor.



Below is an example for how two processes may transition through some of the states as shown in the diagram above:

Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process ₀ initiates I/O
4	Blocked	Running	Process ₀ is blocked,
5	Blocked	Running	so Process ₁ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process ₁ now done
9	Running	–	
10	Running	–	Process ₀ now done

4.3 Data Structures

- The process list will contain all processes that are ready and some extra information for what processes are currently running.
- The OS must also keep track of blocked processes.
- The register context will hold the contents of the registers of a stopped process. The registers of a stopped process are written to memory, and are restored when the process resumes.

Below is an example of some data structures used in the xv6 kernel:

```

1 // the registers will save and restore to stop and restart a process
2 struct context {
3     int eip;
4     int esp;
5     int ebx;
6     int ecx;
7     int edx;
8     int esi;
9     int edi;
10    int edp;
11 };
12
13 // the possible states a process can be in:
14 enum proc_state { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE
15     };
16
17 // the information xv6 tracks about each process
18 struct proc {
19     char *mem; //start of process memory
20     uint sz;
21     char *kstack; // bottom of kernel stack for this process
22     enum proc_state state;
23     int pid;
24     struct proc *parent;
25     void *chan; // !zero -> sleeping on chan
26     int killed; // !zero -> killed
27     struct file *ofile[NOFILE]; // open files
28     struct inode *cwd;
29     struct context context;
30     struct trapframe *tf;
31 }

```

As seen above there are more states that a process can be in than the three listed;

- An initial state for when it is being created
- A final state where it has exited but has not yet been cleaned up (zombie)

5 Interlude: Process API

5.1 The `fork()` System Call

See `p1.c` for an example.

- Once the process reaches the `fork()` syscall, the OS will create a (nearly) identical copy of the calling process, just in its own address space
- From the perspective of the OS, there are two copies of `p1.c` running, each about to return from `fork()`.
- The child does not start at `main()`, but rather the site in which `fork()` was invoked.
- For the parent, `fork()` will return the PID of the child, and for the child it returns 0.
- The output is also not deterministic: either the child or parent may print first. This is determined by the CPU scheduler.

5.2 The `wait()` System Call

See `p2.c` for an example.

- In this example the parent invokes the `wait()` syscall, which blocks it until the child finishes executing.
- This will ensure that the program is now deterministic, as the child will always print first.

5.3 The `exec()` System Call

See `p3.c` for an example.

- On success, `exec()` doesn't return
- It will take the current process and replaces it with a different executable and starts at `main()`.

5.4 Motivating the API

- The separation between `fork()` and `exec()` allows the shell to run code after the `fork()` but before the `exec()`.
- This can alter the environment of the program that is to be executed.
- The shell shows a prompt and waits for the user to type a command.
- It then forks a new child process, calls `exec` to run the program, and waits for the command to finish, in which it prompts the user again.
- The separation between the fork and `exec` allows the shell to implement some useful behaviour, including:

```
1 prompt> wc p3.c > newfile.txt
```

- The `>` redirects the output of `wc p3.c` into `newfile.txt`. This is done by first forking a new child process and then opening `newfile.txt` before executing `wc p3.c`.
- This redirects the output from `stdout` to `newfile.txt`. See `p4.c` for an example.
- `close()` takes a file descriptor as a parameter, and deallocates it (for whichever process called it). Note that it does not stop/terminate any processes or free PIDs.

- The stdout fd is always 1 (defined via `STDOUT_FILENO` and is now free), and `open()` will always return the lowest fd, which is now set to `p4.output`.
- `wc p4.c` then writes to `p4.output` instead of stdout.

For the child process:				
	fd 0	fd 1	fd 2	
after <code>fork()</code>	tty	stdout	tty	(copied from parent)
after <code>close(1)</code>	tty	(free)	tty	
after <code>open(...)</code>	tty	<code>p4.output</code>	tty	<code>open</code> returns lowest fd = 1

- A similar implementation is done for `pipe()`, which lets you connect the output a process to the kernel pipe (or a queue), which connects to the input of another process. For example,

```
1 prompt> grep -o foo file | wc -l
```

- Will get the number of times foo appears in file and pipe it to wc.

5.5 Process Control and Users

- `kill()` is used to send signals to a process, including directives to pause, die and other things.
- On most shells, keystrokes can also send signals (ctrl-c sends `SIGINT`, graceful termination and ctrl-z sends `SIGSTP`, which pauses a program and can be resumed with something like `fg`.)
- A process should also implement the `signal()` syscall to catch different signals; for example, to cleanup resources when a `SIGINT` is issued.
- Modern systems require a strong notion of a "user" to ensure that different people have various levels of access to system resources and signals.

6 Mechanism: Limited Direct Execution

How to efficiently virtualize the CPU with control: The OS must virtualize the CPU in an efficient manner while retaining control over the system. Both hardware and OS support is required.

6.1 Basic Technique: Limited Direct Execution

- A technique used by operating systems to run programs on the CPU while keeping control of system security and resources.
- "Direct execution" refers to the fact that programs run on the CPU

OS	Program
Create an entry for process list	
Allocate memory for program	
Load program into memory	
Setup stack with argc and argv	
Clear registers	
Execute main()	Run main()
	Execute return from main()
Free memory of process	
Remove from process list	

Table 1: Direct execution protol, no limits

- Issues: How do we restrict operations for specific programs? How do we implement time sharing to virtualize the CPU?

6.2 Problem #1: Restricted Operations

How to perform restricted operations: A process must be able to perform I/O and some other (possibly) restricted operations, but without giving the process full access over the system. How does the OS implement this while still executing as fast as possible?

Definition 6.1 (User mode). User mode is a restricted operating state. Code ran in user mode does not have access to certain operations, such as issuing I/O requests. Performing a restricted operation in user mode will cause the processor to raise an exception and the OS to kill the proces.

Definition 6.2 (Kernel mode). The operating state in which the operating system (or kernel) runs in. There are no hardware-enforced restrictions in this mode.

- System calls allow the kernel to expose pieces of kernel functionality to user programs, such as accessing file systems, creating and destroying processes, communicating with other processes and allocating memory.
- System calls are executed via trap instructions, which is a jump + elevation into kernel mode.
- When finished, the OS issues a return-from-trap instruction, which returns back to the user program and reduces priviledges to user mode.
- The kernel controls what code executes during a trap with a trap table (set on boot).

OS @ boot (kernel mode)	Hardware	
Initialize trap table	remember address of syscall handler	
OS @ run (kernel mode)	Hardware	Program (user mode)
Create entry for process list		
Allocate memory for program		
Load program into memory		
Setup user stack with argv		
Fill kernel stack with reg/PC		
return-from-trap	restore regs(from kernel stack) move to user mode jump to main	Run main() /*executing..*/ Call syscall trap into OS
	save regs (to kernel stack) move to kernel mode jump to trap handler (from trap table)	
Handle trap		
Do work for syscall		
return-from-trap	restore regs(from kernel stack) move to user mode jump to PC	/*keep executing...*/ return from main trap invoked by exit()
Free memory of process		
Remove from process list		

Table 2: Limited Direct Execution Protocol

- The trap table contains various exceptions and one entry for syscalls (eg 0x80 on x86).
- When a syscall is invoked, a system-call number is assigned to each unique syscall. This tells the trap handler which syscall to invoke.
- User trap instruction -> hardware trap table -> syscall trap handler -> syscall table (seperate from trap table) -> do stuff

6.3 Problem #2: Switching Between Processes

Assuming a single core system: If a process is running on the CPU, then the OS is not running; how does it do anything?

Specifically, How can the operating system regain control of the CPU so that it can switch between processes?

6.3.1 Cooperative approach: Wait for syscalls

- Most early operating systems
- A cooperative approach means that the OS will assume that long-running processes will periodically give up the CPU so that the OS can do some other stuff.
- This works as most processes frequently issue syscalls to the OS.

- Cooperative operating systems also generally come with a yield syscall which transfers control to the OS so it can do stuff.
- Illegal operations (like dividing by 0, accessing restricted memory) will also generate traps for the OS so that it has control.
- However, this raises the question: How can the OS gain control of the CPU even if processes are not being cooperative?

6.4 Non-cooperative approach

- Timer interrupt: An internal timer will make the hardware periodically issue interrupts to a process every few milliseconds; this will temporarily halt the process and give the OS control through a trap (via an interrupt handler).
- The hardware also has some responsibility during an interrupt: it needs to save enough of the state of the program just before the interrupt so that after the subsequent return-from-trap instruction the program can run correctly.

6.4.1 Saving and Restoring Context

- The scheduler may decide to switch to a different process during an interrupt.
- If this happens, the OS runs a context switch, which saves GPRs, the PC, and the kernel stack pointer for the current process and restores them for the process to be executed.

OS @ boot (kernel mode)	Hardware	
initialize trap table		
	Remember address of syscall handler and timer handler	
start interrupt timer	start timer interrupt CPU in X ms	
OS @ run (kernel mode)	Hardware	Program(user mode)
		Process A /* after X ms... */
	timer interrupt regs(A)->kernel stack(A) move to kernel mode jump to trap handler	
Handle the trap Context switch: regs(A)->proc_t(A) regs(B)<-proc_t(B) switch to kernel stack(B) return from trap to B		
	regs(B)<-kernel stack(B) move to user mode jump to B's PC	
		Process B

Table 3: Limited Direct Execution Protocol (Timer Interrupt)

13 The Abstraction: Address Spaces

13.1 Early Systems

Early OS designs would not abstract away physical address space (1 to 1 mapping).

13.2 Multiprogramming and Time Sharing

- Multiprogramming: When multiple processes are ready to run at a given time, and the OS is responsible for deciding when to switch between them, for example during an IO request.
- This helped increase the effective utilization of the CPUa.
- Time sharing: When the OS allows multiple users or tasks to share the same CPU by switching between them quickly.
- This was initially implemented my letting one process run for a short time (giving it full access to all memory), saving its state to some kind of disk, load the other process's state, and run it.
- The limitation with this implementation is that it is extremely slow; while saving and restoring register-level information is fast, writing memory to disk is very slow.

13.3 The Address Space

The address space is an abstraction of physical memory which contains the memory state of the running program.

- This contains the program code, the heap, and the stack.

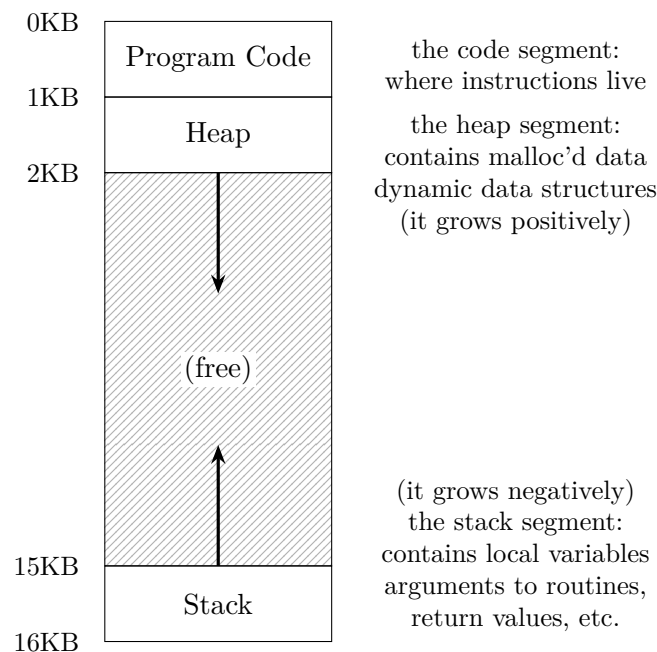


Figure 1: An Example Address Space

See `va.c` for an example of this.

- Note that this is an abstraction from physical memory; the program isn't really loaded in memory at physical address 0 to 16KB.

- This abstracted address is called the virtual address, and the process in which the OS performs this abstraction is virtualizing memory.

13.4 Goals

1. Transparency: The OS should implement virtual memory in a way that is invisible to the running program; i.e. the running program should not be aware that the memory it is accessing is a virtual address.
2. Efficiency: The OS should make virtualization as fast and as space-efficient as possible. This often requires hardware support (eg translation lookaside buffers)
3. Protection: The OS must ensure that that one process cannot access or affect in any way the memory contents of any other processes (that are outside of its address space).

14 Interlude: Memory API

In Unix/C programs, understanding how to allocate and manage memory is critical in building robust and reliable software. What interfaces are commonly used? What mistakes should be avoided?

14.1 Types of Memory

- Stack memory is implicitly managed by the compiler (also known as automatic memory)
- When the variable goes out of its current scope it gets deallocated; thus heap memory is used for long lasting memory

14.2 The malloc() Call

- Allocates memory on the heap, and returns either a pointer to the memory or NULL on fail.

```

1 void *malloc( size_t size ); // defined in header <stdlib.h>
2
3 // the return of malloc should be cast to the type of the underlying
  data.
4 double *d = (double *) malloc(sizeof(double));

```

14.3 The free() Call

This frees memory that has been previously allocated on the heap.

```

1 void free( void *ptr ); // defined in header <stdlib.h>

```

- Note that it is UB if `ptr` does not equal a value returned by one of malloc's variants.
- Using `ptr` after it has been freed is undefined (use-after-free).
- It is undefined if the memory area referred to by `ptr` has already been deallocated (double-free).
- If your program is short-lived, it may not be necessary (albeit bad practice) to free memory, as the address space of a program gets wiped on return.

14.4 Underlying OS Support

- Note that `malloc` and `free` are not syscalls; instead they are implemented by third party libraries (such as `libc`) which act as thin wrappers around syscalls.
- One syscall is `brk`, which changes the location of the end of the program's break or heap.
- `mmap` is another syscall; with the anonymous flag it creates a memory region that is not backed by any file, but is instead associated with swap space. This region can then be treated like a heap and managed as such.

15 Mechanism: Address Translation

How can we build an efficient virtualization of memory?

- The OS needs to have both efficiency and control over the program when virtualizing memory.
- Make use of hardware support whenever possible (registers, TLB, page-tables, etc.)
- Ensure no application is able to access any memory but its own
- *Hardware-based address translation*: Also known as just address translation for short - On any memory access, the hardware transforms the virtual address to the physical location of the data requested.
- It is the hardware's responsibility to translate virtual addresses to the physical locations and enforce protection (bounds and permission checks).
- The OS is responsible for keeping track of occupied and free physical locations, setup, and exception handling.
- This becomes more complicated when you factor in the fact that different program's address spaces may have the same virtual address but map to different physical locations which the OS must keep track of.
- Interposition: Placing a layer in between the software and an interface to add some extra functionality. Interposition (should) be invisible to the caller.
- In the context of virtualizing memory, the hardware interposes every memory access by translating the virtual address to a physical location.

15.1 Assumptions

For now, assume:

- User address space must be contiguous in physical memory.
- User address space is smaller than the size of physical memory.
- Each address space is exactly the same size

15.2 Example

Consider:

```
1 void func() {  
2     int x = 3000;  
3     x = 3 + 3; // line of code we are interested in  
4     /* ... */  
}
```

5 }
}

With no optimizations (x86):

```
1 ; ...  
2 movl -8(%rbp), %eax  
3 addl $3, %eax  
4 movl %eax, -8(%rbp)  
5 ; ...
```

- x is a local variable at address `%rbp - 8`, and gets moved into register `eax`.
- Add the constant 3
- Move x out of `eax` back into its stack address
- From the program's perspective, its address space starts at 0 to some number (say, 16KB).
- The OS needs to place the process somewhere else, not necessarily starting at address 0. How?

15.3 Dynamic Relocation

- Also known as base and bounds, this is a hardware-based technique, where each CPU requires two hardware registers, the base and bounds (or limit) register.
- Each program is compiled as if it starts at address 0.
- When a process is loaded, the OS decides where in physical memory to place the program and sets the base register to this value.
- Once the program starts running, it adds the base register value to get the physical address

$$\text{physical address} = \text{virtual address} + \text{base}$$

- Note that the interposition ensures that no "add base register" ever shows up in the instructions themselves (interposition is invisible)
- Address translation: transforming a virtual address into a physical address
- Since the addressing happens at runtime (base register is set when the program is compiled and loaded), it is dynamic relocation.
- The bounds register is used to ensure that no accesses exceed the program's address space
- Bounds register stores the program size (virtual address of the largest allowed memory access, starting at 0); exception is raised before translation.
- The portion of the CPU that handles address translation is called the memory-management unit (MMU).
- Example: consider a process with an address space of 4KB that is loaded at physical address 16KB.
- Base register is set to 16KB, Bounds register is set to 4KB

Virtual Address		Physical Address
0	→	16KB
1KB	→	17KB
3000	→	19384
4400	→	<i>fault (out of bounds)</i>

15.4 Hardware support: A summary

- The hardware must have support for two different modes: kernel mode and user mode.
- A single bit is usually used for this to indicate which mode the CPU is currently running in.
- Hardware must also provide base and bound registers, plus the MMU of the CPU.

15.5 Operating System Issues

Responsibilities of the OS include:

- OS must find space in physical memory for a newly created process (under our assumptions, this should be trivial).
- The OS will search a **free list** which lists ranges of physical memory not currently in use to find where to place the process.
- The OS must put the memory of a process back on the free list on exit.
- Since each CPU only has one base-bounds register pair, it must save the contents of these registers when context switching into some per-process area such as the process structure or process control block.
- The OS must provide exception handlers; these handlers are configured at boot time. On most exceptions the OS will terminate the process that caused the exception.

OS @ boot	Hardware	No program yet
initialize trap table	Remember addresses of... syscall handler timer handler illegal mem-access handler illegal instruction handler	
start interrupt timer	start timer; interrupt every X ms	
initialize process table		
initialize free list		

16 Segmentation

How do we support a large address space with (potentially) a lot of free space between the stack and the heap?

16.1 Segmentation: Generalized Base/Bounds

- Have multiple base/bounds registers (one per logical segment) in the MMU.
- One segment is a contiguous portion of the address space of a specific length.
- A canonical address space has three segments: the code, stack, and heap.
- Via segmentation the OS places each one of these in separate places in memory to avoid filling physical memory with unused virtual space.
- Stack still grows negatively (convention)
- In this case, the hardware must be able to support three pairs of base/bounds registers.

16.2 Which segment are we referring to?

- Explicit approach: Use the top n-bits of the virtual address to identify which segment the address is for (minimum number of bits to represent number of segments)
- Eg. with three segments, use top 2 bits; 00 -> code segment, 01 -> heap, 11 -> stack.
- Bottom bits are the offset into the segment.
- Would look something like:

```
1 /* ... */
2 // assuming 14-bit VA, 3 segments:
3
4 // get top 2 bits:
5 Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
6 // then the offset
7 Offset = VirtualAddress & OFFSET_MASK
8 // check bounds
9 if (Offset >= Bounds[Segment]) RaiseException(PROTECTION_FAULT)
10 else {
11     PhysAddr = Base[Segment] + Offset
12     Reg = AccessMemory(PhysAddr)
13 }
14 /* ... */
```

- As having two bits allows for 4 segments (resulting in an unused segment), some systems will merge the code and heap segment and use one bit to identify the segment.
- Each segment is also limited in size as the top n bits are reserved.
- Implicit approach involves the processor inferring the segment
 - Generated from PC is in code segment
 - Based off of the stack or base pointer is in stack segment
 - Any other must be in the heap segment

16.3 What about the stack?

- Since the stack grows backwards, we add an extra bit to each base/bounds reg pair that signifies whether it grows positive or negative.
- Subtract size bounds from base bounds instead of adding if grows negative

16.4 Support for sharing

- Being able to share specific code segments can also be useful.
- In order to protect certain parts, protection bits are used to signify which users/programs are allowed to do what (read, write, execute).

16.5 Fine-grained vs Coarse-grained segmentation

- Coarse-grained have few large segments.
- Fine-grained allows for more smaller segments; this requires more hardware support through a segment table.

16.6 OS Support

- On context switch, the OS should save all segment registers to ensure that they are restored to correct values.
- External fragmentation: small gaps of physical memory in between segments
- Excessive fragmentation can cause the OS to struggle to find large enough contiguous blocks of memory when allocating a new process.
- A naive fix is to compact each segment and rearrange them to be contiguous; this is expensive
- Free-list management is generally preferred. Algorithms include best-fit, worst-fit, first-fit, and more complex ones such as buddy algorithm.

17 Free-Space Management

- Managing free space is easy when you have fixed-size units (such as through paging).
- Becomes more difficult through user-level memory allocation (like `malloc()` and `free()`)
- The OS must handle fragmentation in some way

17.1 Assumptions

- A basic interface via `void *malloc(size_t size)` and `void free(void *ptr)`
- The data structure that manages free space in the heap is some kind of free list
- Ignore internal fragmentation
- Allocated memory cannot be relocated (no compaction)
- Allocator manages a contiguous fixed size region of bytes

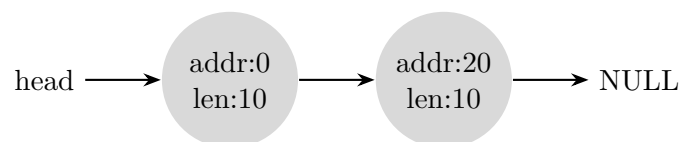
17.2 Low-level mechanisms

17.2.1 Splitting and Coalescing

Consider the following:



The free list for this heap would have two elements and look something like:



- Requesting anything larger than 10 bytes will fail
- The allocator will have to perform splitting to allocate something smaller than 10 bytes
- *Splitting*: Split a segment of free memory into two; one is returned to the caller and the other remains on the free list.

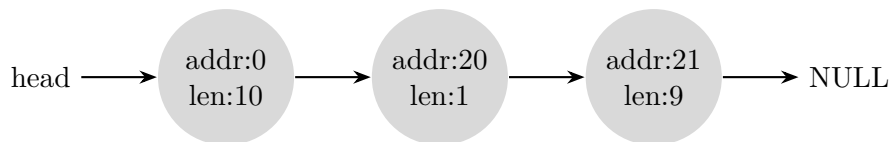


Figure 2: Splitting (step 1)

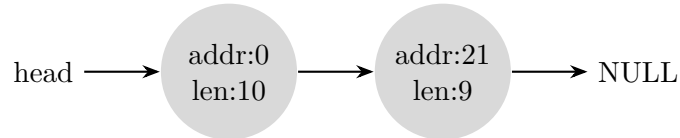


Figure 3: Splitting (result)

- Coalescing: merging contiguous regions of free space

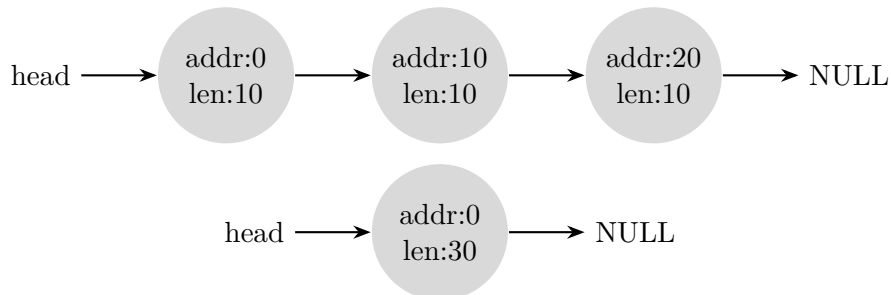
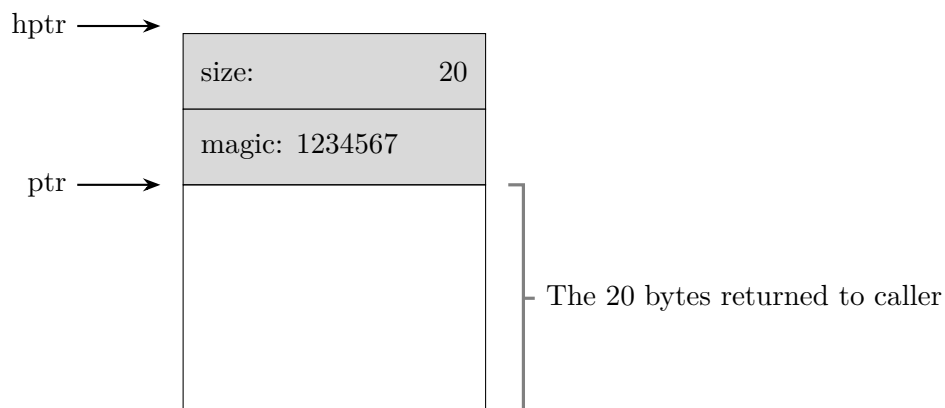


Figure 4: After coalescing

17.2.2 Tracking the size of allocated regions

- Since `free` does not take size as a parameter, the allocator must be able to deduce quickly the size of the region to free
- Each block of allocated memory has a header that contains a magic number (integrity check) as well as how much data was allocated



- When the user calls `free`, the allocator uses pointer arithmetic to figure out where the header begins.

```

1 #define MAGIC 1234567;
2
3 typedef struct {
4     int size;
5     int magic;
6 } header_t;
7
8 void free(void *ptr) {
9     header_t *hptr = (header_t *) ptr - 1;
10    assert(hptr->magic == MAGIC);
11
12    /* ... */
13 }
14
15 typedef struct __node_t (
16     int size;
17     struct __node_t *next;
18 ) node_t;
19 // embed the free list
20 // free list starts with one node of size 4096-bytes
21 node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_ANON|
22     MAP_PRIVATE, -1, 0);
23 assert(head != MAP_FAILED);
24 head->size = 4096 - sizeof(node_t);
25 head->next = NULL;
26 /* ... */

```

17.2.3 Embedding a free list

See above